

Características dos Hooks

Os Hooks transformam componentes funcionais, trazendo estado e ciclo de vida sem classes. Vamos explorar suas características e boas práticas para usá-los corretamente.



Nomenclatura dos Hooks

useState

Gerencia estados dentro de componentes funcionais. Substitui `this.state` e `this.setState` das classes.

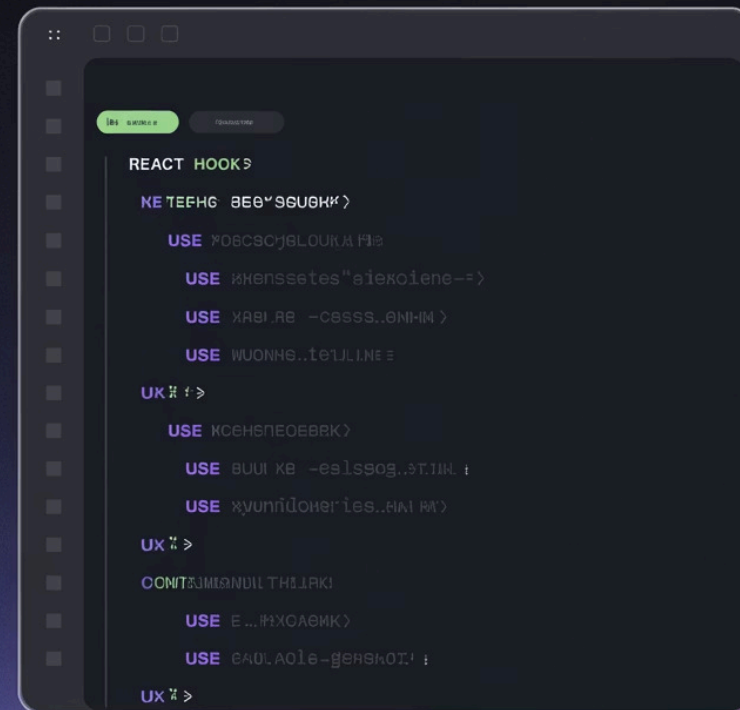
useEffect

Controla efeitos colaterais como ciclo de vida, chamadas API e manipulação do DOM.

useContext

Permite acesso a contextos globais sem componentes consumidores aninhados.

Todo Hook sempre começa com a palavra "use". Isso ajuda desenvolvedores e o React a identificá-los corretamente.

[Dock](#)[Documentation](#)[Tutorials](#)[Community](#)[Download](#)

Unlock the power of React Hooks

Do not worry

Regras de Invocação

```
import { useState } from "react";  
  
const [name, setName] = useState('');
```



No topo do componente

Hooks devem ser chamados na parte superior, onde declaramos variáveis.



Nunca em loops

Não use dentro de estruturas como for ou while.



Nunca em condicionais

Evite chamar Hooks dentro de estruturas if ou switch.



Nunca em funções internas

Não declare Hooks dentro de outras funções no componente.

Essas regras garantem que o React mantenha a ordem e controle corretos dos estados e efeitos.

Adeus this e bind

Componentes de Classe (Antes)

- Uso de this para acessar propriedades
- Necessidade de bind em métodos
- Constructor com inicialização de state
- Mais verboso e complexo

```
class App extends React.Component {
  constructor() {
    super();
    this.state = { count: 0 };
    this.increment = this.increment.bind(this);
  }

  increment() {
    this.setState({ count: this.state.count + 1 });
  }

  render() {
    return <button onClick={this.increment}>Incrementar</button>;
  }
}
```

Componentes Funcionais (Hooks)

- Sem this ou bind
- Estado com useState
- Funções declaradas diretamente
- Código mais limpo e intuitivo

```
export function App() {
  const [count, setCount] = useState(0);

  function increment() {
    setCount(count + 1);
  }

  return <button onClick={increment}>Incrementar</button>;
}
```

Os Hooks eliminam a complexidade das classes, tornando o código mais legível e fácil de manter.